Version 1.1

January 2026

DATA MINING

MODULE 2 - EXPLORATORY DATA ANALYSIS (EDA) & DATA VISUALIZATION

Author:

Yufis Azhar, Ir., S.Kom., M.Kom.

Haidar Dimas Heryanto

Muhammad Aunul Hakim

INFORMATICS LABORATORY

University of Muhammadiyah Malang

TABLE OF CONTENTS

TABLE OF CONTENTS **2**
INTRODUCTION **3**
PRACTICAL OBJECTIVES 3
SOFTWARE / APPLICATION PREPARATION 3
KEYWORDS 3
A. Exploratory Data Analysis (EDA) **4**
1. Dataset 4
What is a Dataset ? 4
Initializing Data Using the Pandas Library 4
Initializing Data From Kaggle 5
Initializing Dataset From Google Drive 7
2. Data Cleaning 7
Handling Missing Value 7
B. Data Visualization **15**
Bar Chart 16
Multi Bar-Chart 17
Line Chart 18
Histogram 19
Scatter Plot 20
CODELAB (20%) **21**
LAB ASSIGNMENT (80%) **21**
Assignment 1 - Exploratory Data Analysis (40%) 22
Assignment 2 - Data Visualization (40%) 22
GRADING CRITERIA & DETAILS **22**

INTRODUCTION

PRACTICAL OBJECTIVES

1. Students are able to understand the purpose and workflow of Exploratory Data Analysis (EDA).
2. Students are able to perform basic data inspection, cleaning, and preprocessing for analysis.
3. Students are able to identify patterns, trends, and anomalies in datasets using EDA techniques.
4. Students are able to create effective data visualizations using Python libraries (e.g., Matplotlib and Seaborn).
5. Students are able to interpret and communicate insights derived from data visualizations.

SOFTWARE / APPLICATION PREPARATION

1. Muslim students may recite the following prayer before starting the lesson

رَضِيْتُ بِاللهِ رَبًّا وَبِالإِسْلَامِ دِيْنًا وَبِمُحَمَّدٍ نَبِيًّا وَرَسُوْلًا. رَبِّ زِدْنِيْ عِلْمًا وَارْزُقْنِيْ فَهْمًا

2. Hardware

- Laptop/Computer

3. Software (Choose one)

- Visual Studio Code, Google Collaboratory, PyCharm, Jupyter Notebook

4. Tutorial

- Using Google Colab : https://youtu.be/tpDjhSzJor4?si=bFjSP_mf3oB5x520
- Using Google Colab Jupyter in VS Code : https://youtu.be/HOg13z1wI1A?si=RhEY2QY5bir-dzEH
- Using Jupyter Notebook : https://youtu.be/HQaCF2F_MRo?si=k6ubuGuCg-VS0tQe
- Install Anaconda : https://youtu.be/XyudQ3B3OWI?si=7qugN_vTxAM30qDt

5. Google Colab Material

MODULE 2 DATA MINING - EDA & Visualization Data.ipynb

KEYWORDS

Exploratory Data Analysis, Preprocessing, Data Visualization

A. Exploratory Data Analysis (EDA)

Data is a collection of information or raw facts that can be symbols, numbers, words, or images. However, raw data is difficult to understand without adequate analysis. Exploratory Data Analysis (EDA) is an exploratory method used to understand the contents of a dataset through statistical summaries and graphical visualizations. EDA aims to find patterns, trends, and relationships between variables. Thus, data can be evaluated in depth to provide useful insights before proceeding to further analysis or building predictive models.

Data visualization is an important step in EDA, which is the process of displaying data in the form of graphs or diagrams to make it easier to understand. Data visualization helps to reveal patterns hidden in raw numbers, allowing users to see trends and relationships more intuitively. Some types of graphs that are often used include histograms for data distribution, bar charts for category comparisons, and pie charts for data proportions. Libraries that facilitate data processing and data visualization include **Matplotlib, Seaborn**, and **Pandas**.

1. Dataset

What is a Dataset ?

A dataset is a collection of data that is organized in a structured manner and stored in a specific format, such as a table or file. Datasets can contain numerical data, text, images, or a combination of all of these. You can collect data manually or access datasets from online platforms such as Kaggle, UCI Machine Learning Repository, Google Dataset Search, and others.

Initializing Data Using the Pandas Library

In Python, data initialization can be done in various ways depending on the type and structure of the data required. One way is to initialize a dataset using the pandas library in Python.

To make it more interactive and allow you to experiment with the material in module 2, access the following dataset, then download the fastfood.csv file and upload it to your Colab.

Here's how to read a dataset from a CSV file:

```
import pandas as pd

# Inisialisasi DataFrame dari file CSV
df = pd.read_csv('fastfood.csv') # path file dataset
# df = pd.read_excel('fastfood.xlsx')
```

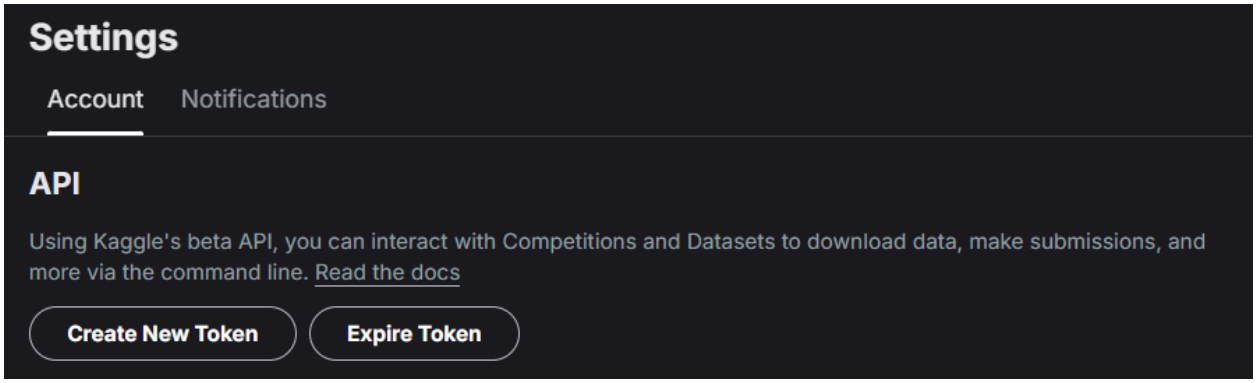
```
# df= pd.read_excel
df
```

Initializing Data From Kaggle

As explained earlier, datasets can be accessed from online platforms, one of which is Kaggle. Kaggle is an online platform that provides resources for data science and machine learning. This platform allows users to find, publish, and use data sets, as well as participate in data science competitions.

Dataset can vary in terms of data volume and type. It would take up a lot of storage space and time if we always downloaded datasets from online platforms such as Kaggle. But don't worry, we can access datasets from Kaggle without having to download them.

First, make sure you already have a Kaggle account. Next, go to the My Account section and scroll down until you find the API as shown in the following image.



Click the create token button and it will automatically download kaggle.json, which contains credential information for accessing the Kaggle API on your local device.

Then, on Google Colab/Jupyter Notebook, install the Kaggle library.

```
# Install kaggle
!pip install kaggle
```

Upload the kaggle.json file to your colab. After that, make sure the kaggle.json file is in the correct directory, use the following command:

```
# Copy kaggle.json to Folder /root/.kaggle/ So Kaggle CLI can be access.
!mkdir /.kaggle
```

```
!mv kaggle.json /.kaggle
!mv /.kaggle /root/
!chmod 600 ~/.kaggle/kaggle.json
```

And yes, after setting up your Kaggle account and connecting Colab to Kaggle, you can now access datasets from Kaggle without the hassle of downloading them. To download a dataset from Kaggle, use the following command

```
!kaggle datasets download -d <path-to-dataset> -p <download-location>
```

Let's take an example. Access the following dataset at Kaggle. You will see a download button at the top right. Click the button, then download via Kaggle CLI and copy the command provided, as shown in the following code.

```
#!/bin/bash
!kaggle datasets download ulrikthygepedersen/fastfood-nutrition
!unzip fastfood-nutrition.zip
```

But if you want to use KaggleHub, you can copy the code below.

```
import kagglehub
import shutil
import os

# 1. Download dataset
path =
kagglehub.dataset_download("ulrikthygepedersen/fastfood-nutrition")
print("Original location:", path)

# 2. Set the target location in /content
target_path = "/content/fastfood-nutrition"

# 3. Copy the contents of the cache folder to /content
if not os.path.exists(target_path):
    shutil.copytree(path, target_path)
    print(f"Dataset successfully copied to: {target_path}")
else:
    print("Folder already exists.")
```

The downloaded dataset will be saved to your local Colab in a .zip folder. Then, unzip the file using the command above. That way, you don't have to download the dataset manually and waste your storage space 😀🎉.

Initializing Dataset From Google Drive

If you need to download the dataset, you can save it to your Google Drive account. To initialize it, use the following code:

```
from google.colab import drive
drive.mount("/content/drive")
```

Then, make sure you have uploaded the dataset to Google Drive. Your Google Colab account and Google Drive account must be the same account. To read the dataset that will be used, copy the dataset path and then use pandas to read your dataset.

```
df_from_drive=pd.read_csv('/content/drive/MyDrive/Dataset/fastfood.c
sv') # Change the path according on your Google Drive path
```

This way, if you exit colab and need to rerun it, you don't have to manually upload the dataset repeatedly.

2. Data Cleaning

Data cleaning is the process of detecting and correcting inaccurate or unusual data from data sets, tables, or databases to identify incomplete, incorrect, inaccurate, irrelevant, or messy data, which is then modified, replaced, or deleted as needed.

There are various types of data that are often encountered. Some data is clean and ready for processing, but some data is dirty, with indications of missing data, data values that do not match the data type, and duplicate data. The existence of dirty data will have a big impact on your analysis results, and will often cause misinterpretation of results when performing aggregation.

Handling Missing Value

Missing values are a common occurrence when processing data, due to unavailable data resulting in some missing data. Missing values can be detected by the appearance of NaN (Not Available) values or blank spaces in a table. Let's try it.

```
import pandas as pd
df = pd.read_csv('/content/fastfood-nutrition/fastfood.csv') #
Initialize df with the downloaded dataset
df.isnull() # Check for NaN Data
```

```
df.isna()
```

Output :

	restaurant	item	calories	cal_fat	total_fat	sat_fat	trans_fat	cholesterol	sodium	total_carb	fiber	sugar	protein	vit_a	vit_c	calcium	salad
0	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False
1	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False
2	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False
3	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False
4	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
510	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False
511	False	False	False	False	False	False	False	False	False	False	False	False	False	True	True	True	False
512	False	False	False	False	False	False	False	False	False	False	False	False	False	True	True	True	False
513	False	False	False	False	False	False	False	False	False	False	False	False	False	True	True	True	False
514	False	False	False	False	False	False	False	False	False	False	False	False	False	True	True	True	False

515 rows × 17 columns

We can see from the output above that if the value is False, then the data is not NaN, and vice versa. If the value of a piece of data is True, then the data is NaN or a missing value.

To display the number of NaN data, we will add the sum() syntax as in the following example. The numbers in each column display the number of missing values in the dataset column.

```
df.isnull().sum()  # Calculate Total NaN Data
```

Output :

fiber	12
sugar	0
protein	1
vit_a	214
vit_c	210
calcium	210

From the output above, we get the information that some columns have NaN/missing values. Here are some things we can do.

a. Removing or Deleting Data

```
df.dropna(axis=0 , inplace=True)
```

The dropna() function will remove data that has NaN values. The axis=0 parameter specifies that rows with NaN values will be removed, while axis=1 will remove columns that have NaN values. Furthermore, the inplace=True parameter ensures that changes are made directly to the original DataFrame without the need to save the results to a new variable, while the default value **inplace=False** will only create a copy of the modified DataFrame without changing the original DataFrame.

```
# From the dataset above, we will try to delete data that we do not
need for analysis

df.dropna(subset=['fiber'],  inplace=True)  #  subset  is  used  to
determine the columns to be checked
```

Next, we can remove unnecessary columns using the following syntax

```
df.drop(['column_name'], axis=1)
```

Deleting columns with the most missing values will make it easier for us to analyze the data. However, if we need those columns for analysis, should we still delete them? Of course not.

b. Filling in NaN data using Actual Data

Instead of deleting data in a column, we can input values that resemble the minimum value, maximum value, mean, or mode. Consider the following steps. First, we display the statistical description of the dataset.

```
df.describe()
```

Output :

	calories	cal_fat	total_fat	sat_fat	trans_fat	cholesterol	sodium	total_carb
count	503.000000	503.000000	503.000000	503.000000	503.000000	503.000000	503.000000	503.000000
mean	524.453280	234.168986	26.075547	7.983101	0.453280	71.123260	1234.870775	45.447316
std	280.173625	163.384648	18.074278	6.273975	0.827385	61.979954	689.702021	25.082190
min	20.000000	0.000000	0.000000	0.000000	0.000000	0.000000	15.000000	0.000000
25%	330.000000	120.000000	14.000000	4.000000	0.000000	35.000000	800.000000	28.000000
50%	480.000000	200.000000	23.000000	7.000000	0.000000	60.000000	1100.000000	44.000000
75%	680.000000	310.000000	34.000000	10.000000	1.000000	90.000000	1540.000000	56.000000
max	2430.000000	1270.000000	141.000000	47.000000	8.000000	805.000000	6080.000000	156.000000

Using the **fillna()** function, we can fill columns in a DataFrame that have NaN values.

```
df['column_name'].fillna(int(df['column_name'].mean()),inplace=True)
```

```
# From the dataset above, we will assign values to NaN for each
column.
for col in ['fiber', 'vit_a', 'vit_c', 'calcium']:
    df[col] = df[col].fillna(int(df[col].mean()))

# Displaying the data after assigning values.
df.isnull().sum()
```

Output :

fiber	0
sugar	0
protein	0
vit_a	0
vit_c	0
calcium	0

Voilaaa, the data that previously had hundreds of NaN values is now free of NaN values and we can proceed to the next step.

For your information, we can also manually input values with actual data or custom values based on business assumptions. However, this requires a lot of time and effort to find the missing data and input it one by one.

3. Data Duplication

Data duplication occurs when there are repeated values or IDs in a data set. In addition, data is considered duplicate if there is the same information in every column of the data. This duplication problem needs to be addressed through data cleaning, as it will cause double counting when conducting a more in-depth analysis. The first step is to check whether there is duplication in the data, which can usually be indicated by the appearance of the same value in all columns. If your data contains columns with unique information that should not be duplicated, you can check for data duplication using the following syntax.

To check the amount of duplicate data:

```
df.duplicated().sum()
```

To check for duplicate data in all columns:

```
df[df.duplicated(keep=False)]
```

To check for duplicate data in a specific column:

```
df[df.column_name.duplicated(keep=False)].sort_values('column_name')
```

```
# Let's check if the fastfood dataset has duplicate data.

df.duplicated().sum() # Check the number of duplicate data.

df[df.duplicated(keep=False)] # Check duplicate data in all columns.
```

Output :

	restaurant	item	calories	cal_fat	total_fat	sat_fat	trans_fat	cholesterol	sodium	total_carb	fiber	sugar	protein	vit_a	vit_c	calcium	salad
414	Taco Bell	Chili Cheese Burrito	380	150	17	8.0	1.0	35	930	41	5.0	2	16.0	18.0	20.0	24.0	Other
492	Taco Bell	Chili Cheese Burrito	380	150	17	8.0	1.0	35	930	41	5.0	2	16.0	18.0	20.0	24.0	Other
497	Taco Bell	Express Taco Salad w/ Chips	580	260	29	9.0	1.0	60	1270	59	8.0	7	23.0	18.0	20.0	24.0	Other
511	Taco Bell	Express Taco Salad w/ Chips	580	260	29	9.0	1.0	60	1270	59	8.0	7	23.0	18.0	20.0	24.0	Other

Duplication issues can be resolved by deleting duplicate rows, leaving only unique values to be analyzed. To eliminate data duplication, use the following syntax

To remove duplicate data in a single row of a dataframe

```
df.drop_duplicates(keep='first', inplace=True)
```

Keep parameter:

- keep='first': Keeps the first row of duplicates and removes the rest (default).
- keep='last': Keeps the last row of duplicates and removes the rest.
- keep=False: Deletes all duplicate rows.

Next, to remove duplicate data in a specific column

```
df.drop_duplicates(subset=['column_name']).reset_index()
```

```
df.drop_duplicates(keep='first', inplace=True)

df.duplicated().sum()
```

Output : ***np.int64(0)***

4. Data Manipulation

Data manipulation is the process of manipulating or changing data so that it is easy to organize and easy to read/understand. Data manipulation is very helpful in the data analysis process. There are several examples of data manipulation applications, which will be explained as follows.

a. Checking and Changing Data Types

The dtypes and astype() functions in Pandas can be used to check and change data types. This can make it easier to analyze data.

Use the following syntax to check the data type of a column

```
df.<column name>.dtypes
```

To change the data type in a column, you can use the following syntax:

```
df['new_column_name'] = df['column_name_to_be_changed'].astype(data_type)
```

b. Renaming Columns

To change column names as shown in the example below. This is necessary if the column names are difficult to understand in context or are too long/short. Below is the code for changing column names in DataFrame:

```
df.rename(columns={'A': 'X', 'B': 'Y', 'C': 'Z'}, inplace=True)
```

c. Deleting Data

Not all data is used in the analysis. The more data used, the more difficult it will be for us to analyze, so we need to delete some data that we don't use. To reduce data, you can use the drop command as follows:

```
df.drop('Z', axis=1, inplace=True)
```

5. Handle Outliers

An outlier is an observation in a data set that has a different pattern from the other observations in that data set. This means that there are values that are much smaller or much larger than the other data in the data set. The causes of outliers include:

- Errors in the data input procedure
- Errors in measurement or analysis
- Special circumstances Outlier

So, why is it important to identify outliers? Outliers can affect the mean of the data and make the analysis results inaccurate. In addition, we can check whether there are errors in the data that need to be corrected. Sometimes, outliers are important data that represent extreme situations (e.g., very high-calorie foods).

Here are some ways to detect outliers:

a. Boxplot

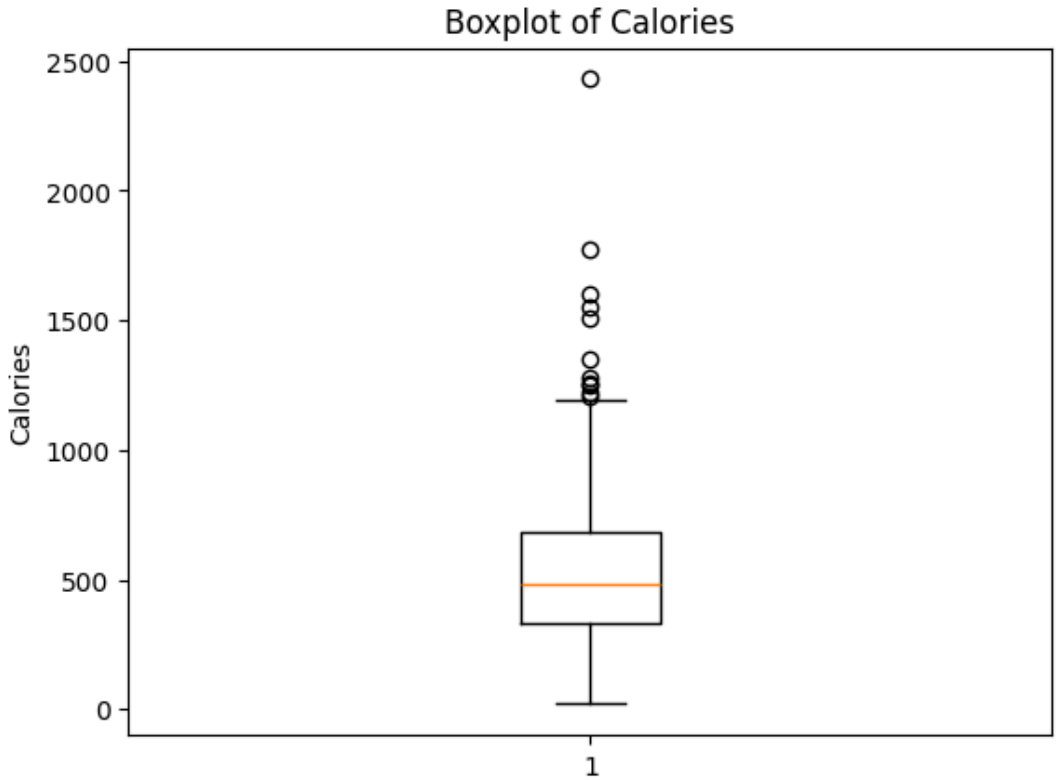
Boxplots are an effective visual method for identifying outliers. Outliers are often displayed as points outside the boxplot's “whiskers.”

Let's try displaying outliers in one of the DataFrame columns.

```
import matplotlib.pyplot as plt

plt.boxplot(df['calories'])
plt.title('Boxplot of Calories')
plt.ylabel('Calories')
plt.show()
```

Output :



We can analyze the visualization results above as follows:

1. The middle box represents data between the first quartile (Q1) and the third quartile (Q3). The line inside the box is the median. The calories data shows that the majority of calorie values are between 250 and around 750.
2. The whiskers (horizontal lines) show the "normal" data range without outliers. The calorie data shows that the upper limit of normal data is around 1000.
3. Points above or below the whiskers are **outliers**. These points are outside the normal range (smaller than the lower bound or larger than the upper bound). In the visualization above, the outliers are above 1000. This means that there are foods with calories ranging from 1500 to 2500, which are much higher than the other data points.

b. Z-Score

Z-Score measures how far an observation is from the mean, in standard deviation units. In other words, z-score shows whether a value is:

- Above average (positive z-score).
- Below average (negative z-score).
- Equal to average (z-score = 0)

By using Z-Score, we can detect outliers in data. Data with z > 3 or z < -3 are often considered outliers. Z-Score transforms data into a distribution with a mean of 0 and a standard deviation of 1, making it suitable for machine learning algorithms.

```
from scipy import stats
import numpy as np

z_scores = np.abs(stats.zscore(df['calories']))
outliers = np.where(z_scores > 3)

print("Outliers index:", outliers)
```

Output : ***Outliers index: (array([38, 39, 44, 47, 190]),)***

c. Interquartile Range (IQR)

Another way to detect outliers is by using the Interquartile Range (IQR). IQR focuses on data quartiles, which divide data into four equal parts

$$IQR = Q3 - Q1$$

- Q1 (First Quartile): The value below which 25% of the data lies.
- Q3 (Third Quartile): The value below which 75% of the data lies.
- IQR: The distance between Q3 and Q1, indicating the main data range.

We can use IQR to detect outliers if the data has a non-normal distribution. IQR is effective for detecting data that is far outside the main range.

B. Data Visualization

There are many types of data visualization, from simple to complex. Each type of visualization is usually more appropriate for a specific purpose. Broadly speaking, there are four general purposes of visualization.

1. **Comparison**

If you want to compare a set of values with other values, to see which is greater, which is higher, which is increasing, etc. For time-related data, the visualization used is

usually a line chart. Meanwhile, if the comparison is categorical, then a bar chart is more suitable.

2. **Distribution**

If you want to see the distribution/spread of data in a variable, the visualization commonly used is a histogram. You can see whether the data is concentrated in small values, weighted to the right, or symmetrical, or whether it is normally distributed. To see the distribution in two variables, a scatterplot can also be used.

3. **Composition**

Used to see the composition of a variable when broken down by a data dimension. The visualization commonly used is a stacked bar chart for categorical data, or a stacked line chart for time-related data.

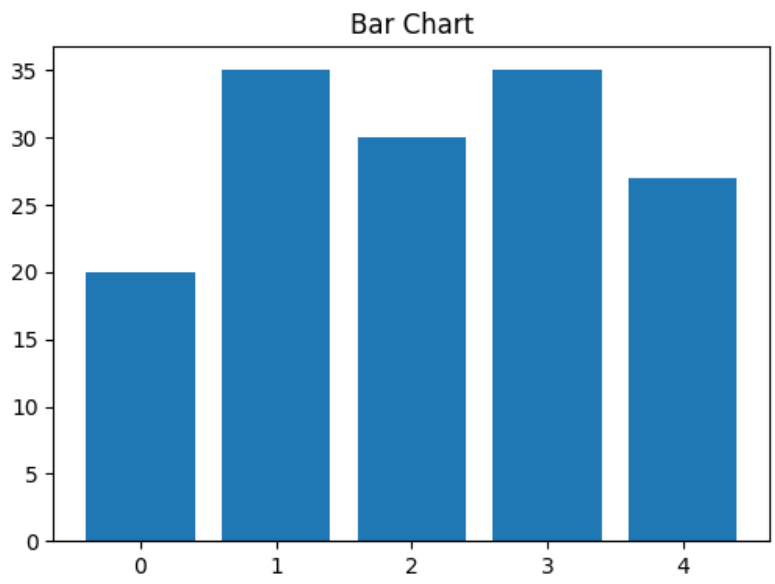
4. **Relationship**

Used to see the relationship between one variable and another. For example, to check whether variable A increases as variable B increases, or whether variable B decreases as variable A increases, or whether there is no relationship between the two variables. The visualization commonly used for this purpose is a scatter plot.

Bar Chart

```
import matplotlib.pyplot as plt
import numpy as np
data_bar = [20, 35, 30, 35, 27]
plt.figure(figsize=(13, 9))
plt.subplot(2, 2, 1) # row 2, col 2, position 1
plt.bar(range(len(data_bar)), data_bar)
plt.title('Bar Chart')
```

Output :
Text(0.5, 1.0, 'Bar Chart')



Multi Bar-Chart

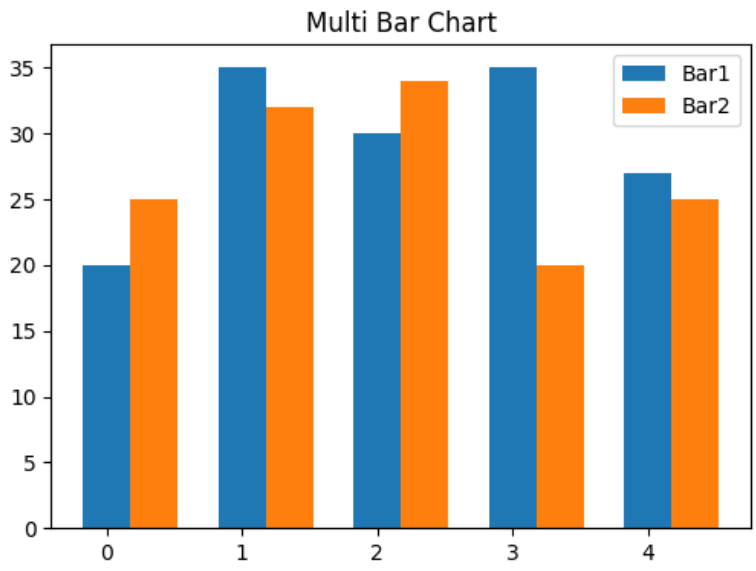
```
import matplotlib.pyplot as plt
import numpy as np

multi_data_bar = {'Bar1': [20, 35, 30, 35, 27],
                  'Bar2': [25, 32, 34, 20, 25]}

plt.figure(figsize=(13, 9)) # Determine the visualization size
plt.subplot(2, 2, 2) # row 2, col 2, position 2
bar_width = 0.35
index = np.arange(len(multi_data_bar['Bar1']))

plt.bar(index, multi_data_bar['Bar1'], bar_width, label='Bar1')
plt.bar(index + bar_width, multi_data_bar['Bar2'], bar_width,
  label='Bar2')
plt.title('Multi Bar Chart')
plt.legend()
plt.show()
```

Output :



Line Chart

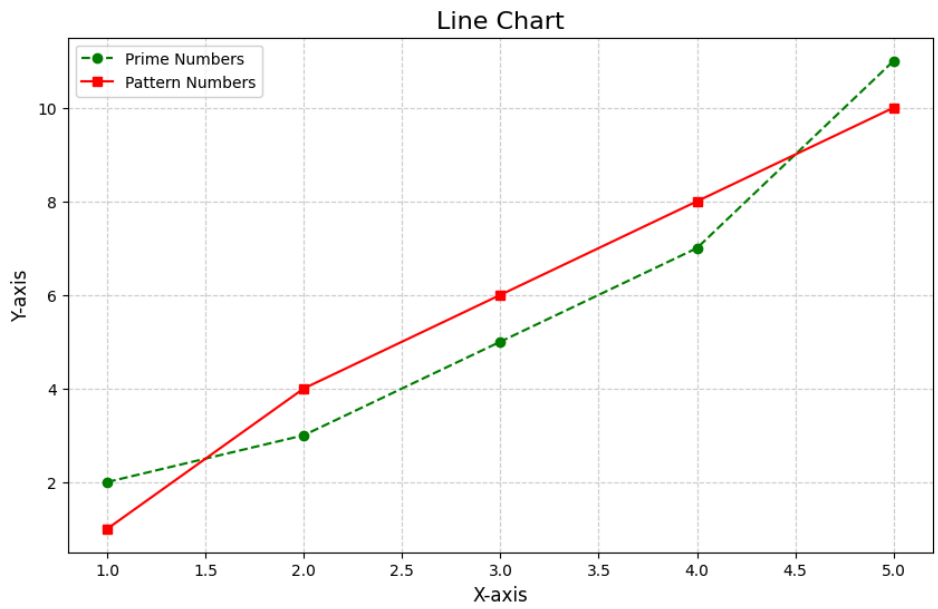
```
import matplotlib.pyplot as plt

# Data
x = [1, 2, 3, 4, 5]
y1 = [2, 3, 5, 7, 11]
y2 = [1, 4, 6, 8, 10]

plt.figure(figsize=(10, 6))
plt.plot(x, y1, label='Prime Numbers', color='green',
         marker='o', linestyle='--')
plt.plot(x, y2, label='Pattern Numbers', color='red',
         marker='s')
plt.title('Line Chart', fontsize=16)
plt.xlabel('X-axis', fontsize=12)
plt.ylabel('Y-axis', fontsize=12)
plt.grid(True, linestyle='--', alpha=0.6)
plt.legend(fontsize=10)

plt.show()
```

Output :



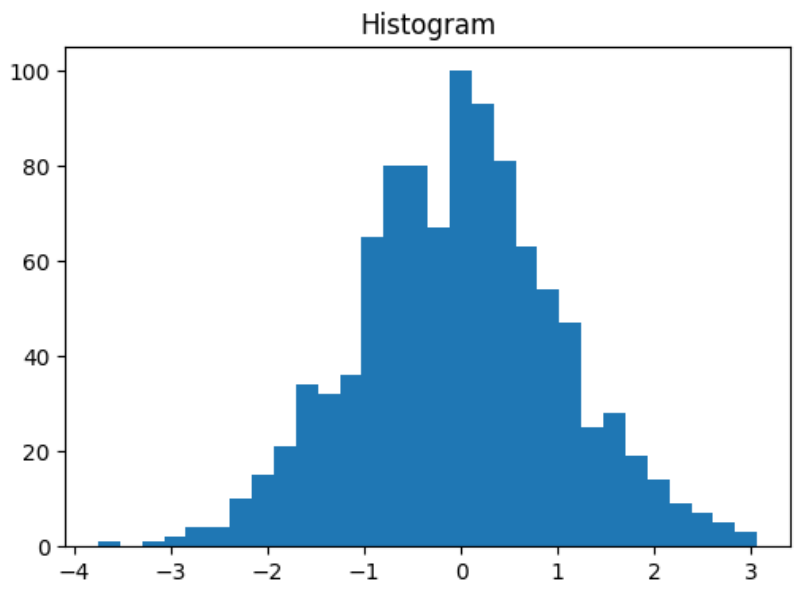
Histogram

```
import matplotlib.pyplot as plt
import numpy as np

hist_data = np.random.normal(0, 1, 1000) # Generate random numbers
  following a normal distribution

plt.figure(figsize=(13, 9))
plt.subplot(2, 2, 3)
plt.hist(hist_data, bins=30)
plt.title('Histogram')
```

Output :



Scatter Plot

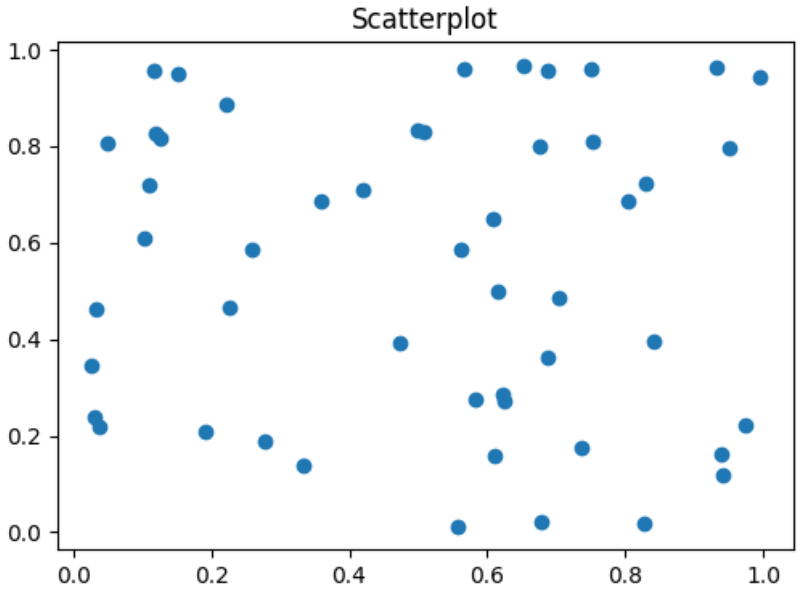
```
import matplotlib.pyplot as plt
import numpy as np

scatter_data_x = np.random.rand(50)
scatter_data_y = np.random.rand(50)

plt.figure(figsize=(13, 9))
plt.subplot(2, 2, 4) # row 2, col 2, position 4
plt.scatter(scatter_data_x, scatter_data_y)
plt.title('Scatterplot')
```

Output :

Text(0.5, 1.0, 'Scatterplot')



CODELAB (20%)

Access the Dataset **here.**

Then perform basic insights from the dataset.

1. **Load the dataset using the pandas library into a dataframe. (25 Point)**
2. **Display the first 7 rows of data and display basic information about the dataset.(25 Point)**
3. **Calculate the total contributions and determine the average contribution of all members. (25 Point)**
4. **Create 2 Visualization (25 points)**

LAB ASSIGNMENT (80%)

Dataset:

- **Odd NIM Endings (1,3,5,7,9): Cafe Sales**
- **Even NIM Endings (0,2,4,6,8): Music Tours**

Every data analyst has their own way of achieving their analysis goals. Using the dataset provided, try analyzing the data above and create a data visualization so that the client can understand the key points of the data you have processed.

Assignment 1 - Exploratory Data Analysis (40%)

1. Display complete dataset metadata accompanied by a brief explanation. (10 Point)
2. Process data (data cleaning and data manipulation).(20 Point)
3. Analyze business problems/questions (minimum of 5). (30 Point)
4. Display descriptive statistical analysis/data outliers. (15 Point)
5. Answer a few questions from your Assistant. (25 Point)

Assignment 2 - Data Visualization (40%)

Create data visualization based on categories, which can be comparisons / relationship / distribution :

1. Create visualizations with a total of 3 different graphs. (40 Point)
2. Explain the visualization results to the assistant. (60 Point)

***If it does not meet the assignment criteria and the source code/analysis steps are exactly/similar to those of other students, then the score will be reduced, even to a maximum score of D.**

GRADING CRITERIA & DETAILS

Activity	Description	Weight
Codelab		**20%**
Lab Assignment	**Exploratory Data Analysis (EDA)**	**40%**
	Data Visualization	**40%**
Sub Total		**80%**
Total		**100%**